

Shelly Zigbee ESPhome notes

Why did I do this?

When Shelly launched their Gen 4 relays with zigbee, I was genuinely excited. I prefer Zigbee devices because they are inherently local. They can never be paywalled, locked out, feature reduced, or cancelled.

When I finally got one, I learned the truth. They work fine on Zigbee, but firmware updates (At least through ZHA) were gated to Wifi. That's right... If you want new firmware, you need to run two protocols. If you turn off Wifi and change your mind later, your only recourse is to reach inside a live electrical box and press the relay button.

I wanted a Zigbee only device with a ZHA upgrade path. I settled on ESPhome because its simple and recently adopted more comprehensive zigbee controls. I got my Shelly 1 Gen 4 paired with ZHA and the relay works in attached mode.

This project was inspired by automatus-io's fantastic Shelly Matter over Thread repo, <https://github.com/automatus-io/shelly-1-gen4-matter-thread>. Shelly documentation for this relay is here, <https://kb.shelly.cloud/knowledge-base/shelly-1pm-gen4>

Build the Flashing harness

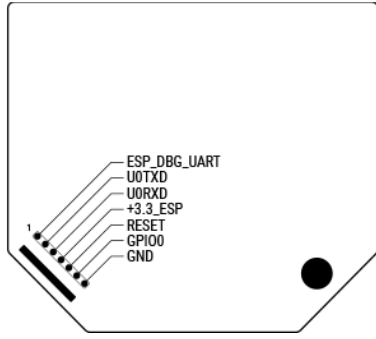
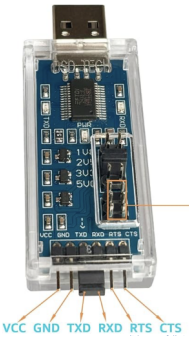
90% of the work in this process is backing up the stock firmware and building a flashing harness. All the parts, links, and prices are in the table below. If you are reading this, I'll bet you already have most of them on hand.

Flashing harness parts:

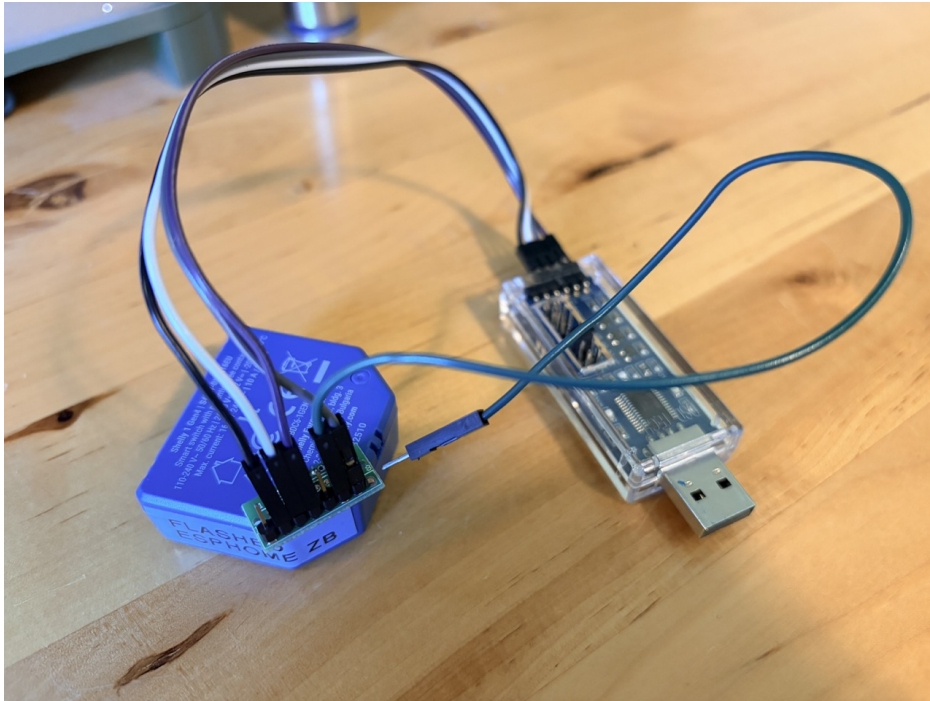
#	Part	Sample Link	Price (10/2026)
1	USB to UART converter	https://www.amazon.com/dp/B07WX2DSVB?ref=ppx_yo2ov_dt_b_fed_asin_title	\$15
2	1.27mm to 2.54mm pin pitch adapter	https://www.amazon.com/dp/B09DGJVST6?ref=ppx_yo2ov_dt_b_fed_asin_title	\$7
3	1.27mm and 2.54mm pin headers	https://www.amazon.com/dp/B0CTKCWNGK?ref=ppx_yo2ov_dt_b_fed_asin_title&th=1 https://www.amazon.com/HiLetgo-20pcs-2-54mm-Single-Header/dp/B07R5QDL8D/ref=sr_1_3	~\$10
4	Dupont cables	https://www.amazon.com/Elegoo-EL-CP-004-Multicolored-Breadboard-arduino/	\$7

The Shelly 1 Gen 4 has 1.27mm pitch pins while USB to UART adapters have 2.54mm pitch pins. I had a cheap adapter board, but I've seen sewing pins, 3d printed pogo pin alignment boards, and many other ways to get this to work. You need a secure way to connect 4 pins from Shelly to the USB adapter and a quick reversible way to bridge the Shelly's GPIO00 Pin to Ground to enter bootloader mode.

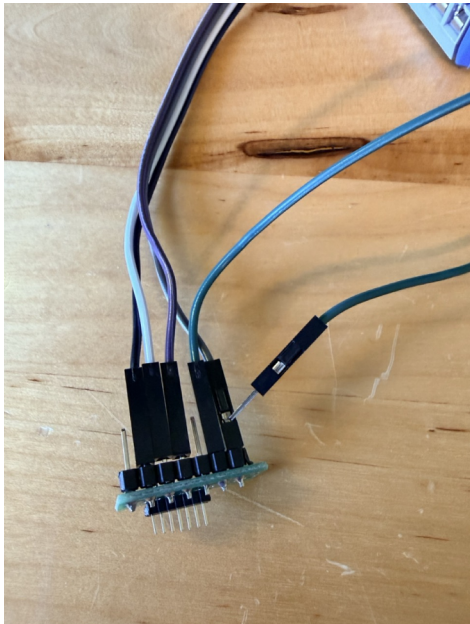
Pinout

Shelly 1 Gen4	USB to UART
Pin 1 (ESP_DBG_UART)	-
Pin 2 (TXD)	RXD
Pin 3 (RXD)	TXD
Pin 4 (3.3V)	3.3V
Pin 5 (RESET)	-
Pin 6 (GPIO0 - BOOT)	-
Pin 7 (GND)	GND
 Diagram showing the pinout for the Shelly 1 Gen4. The pins are labeled from 1 to 7: ESP_DBG_UART, U0TXD, U0RXD, +3.3_ESP, RESET, GPIO0, and GND. A black dot is shown on the right side of the diagram.	 Photo of a USB to UART adapter board. The pins are labeled: VCC, GND, TXD, RXD, RTS, and CTS. The board is blue and has a USB connector at the top.

Here is what mine looked like:



I used a female to male dupont cable to bridge GPIO)) to ground to enter bootloader mode during flashing



Here is another angle on the setup



OEM Firmware Backup:

The process to backup the stock firmware and flash the new device is well covered here, <https://github.com/automatous-io/shelly-1-gen4-matter-thread/blob/main/docs/FLASHING.md>

Here are some notes that can supplement some of the less clear points:

1. The OEM firmware is locked to the Mac address of the relay. Firmware from another relay can not be used
2. Remember to put the Shelly into boot loader mode by grounding GPIO00 (Male dupont connector)
3. Get Port number (Mac Terminal)

```
ls /dev/cu.*
```

- a. On the Mac mini, the port was `/dev/cu.usbserial-BH010DUA`

4. Get mac address (Mac Terminal)

```
python3 -m esptool --port /dev/cu.usbserial-BH010DUA read_mac
```

5. Output on Shelly 1 Gen 4 Blue

```
esptool.py v4.12.0
Serial port /dev/cu.usbserial-BH010DUA
Connecting....
Detecting chip type... ESP32-C6
Chip is ESP32-C6 (QFN40) (revision v0.2)
Features: Wi-Fi 6, BT 5 (LE), IEEE802.15.4, Single Core + LP Core, 160MHz, Embedded
Flash 8MB
Crystal is 40MHz
MAC: <redacted>
BASE MAC: <redacted>
MAC_EXT: ff:fe
Uploading stub...
Running stub...
Stub running...
MAC: <redacted>
BASE MAC: <redacted>
MAC_EXT: ff:fe
Hard resetting via RTS pin...
```

6. Read firmware with esptools

```
esptool.py --chip esp32c6 --port /dev/cu.usbserial-BH010DUA --baud 115200 \
  read_flash 0x0 ALL shelly-1-gen4-stock-<redacted>.bin
```

Had to change Slashes that changed to colons back to dashes

7. Verify with esptools

```
esptool.py --chip esp32c6 --port /dev/cu.usbserial-BH010DUA --baud 460800 verify_flash
0x0 ~/shelly-1-gen4-stock-<redacted>.bin
```

8. Sample Output. Look for “digest matched”

```
esptool.py v4.12.0
Serial port /dev/cu.usbserial-BH010DUA
Connecting....
Chip is ESP32-C6 (QFN40) (revision v0.2)
Features: Wi-Fi 6, BT 5 (LE), IEEE802.15.4, Single Core + LP Core, 160MHz, Embedded
Flash 8MB
Crystal is 40MHz
MAC: <redacted>
BASE MAC: <redacted>
MAC_EXT: ff:fe
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
Configuring flash size...
```

```
Verifying 0x800000 (8388608) bytes @ 0x00000000 in flash against  
/Users/<redacted>/shelly-1-gen4-stock-<redacted>.bin...  
-- verify OK (digest matched)  
Hard resetting via RTS pin...
```

ESPHome Code

Zigbee implementation: In Q426, zigbee in EspHome is an evolving standard. I chose to use the older but proven luar123's external component (https://github.com/luar123/zigbee_esphome). Its "basic mode" maps a switch to the standard on_off cluster, which ZHA shows as a proper switch.

EspHome has native Zigbee component in development (as of Q426) where cluster: on_off is supported. When it eventually arrives in an upcoming release, this code can become a bit simpler.

Here is the code I used:

```
esphome:  
  name: shelly-1-gen4-zigbee  
  friendly_name: Shelly 1 Gen4 Zigbee  
  
esp32:  
  variant: esp32c6  
  flash_size: 8MB  
  toolchain: platformio  
  framework:  
    type: esp-idf  
  
external_components:  
  - source: github://luar123/zigbee_esphome  
    components: [zigbee]  
  
logger:  
  level: INFO  
  hardware_uart: UART0      # logs on the programming header  
  
zigbee:  
  id: zb  
  router: true  
  power_supply: 1  
  components: all  
  
switch:  
  - platform: gpio  
    id: relay  
    name: "Relay"
```

```

pin: GPIO5
restore_mode: RESTORE_DEFAULT_OFF

binary_sensor:
  # SW terminal: internal (no name)
  - platform: gpio
    id: sw_input
    pin: GPIO10
    filters:
      - delayed_on_off: 50ms
    on_state:          # rocker switch; use on_press for a momentary button
    then:
      - switch.toggle: relay

  # Onboard button: short press toggles, hold 5-15 s to leave/rejoin network
  - platform: gpio
    id: device_button
    pin:
      number: GPIO4
      inverted: true
      mode: INPUT_PULLUP
    on_click:
      - min_length: 50ms
        max_length: 1s
        then:
          - switch.toggle: relay
      - min_length: 5s
        max_length: 15s
        then:
          - zigbee.reset: zb

status_led:
  pin:
    number: GPIO0
    inverted: true

```

Flashing the ESPHome firmware

I used EspConnect per the direction here, <https://github.com/automatous-io/shelly-1-gen4-matter-thread/blob/main/docs/FLASHING.md>. But, EspHome Web, <https://web.esphome.io/> would work too.

Pairing with ZHA

1. ZHA adoption took about 30 seconds
2. Entities exposed: On/Off switch for the relay. The PM version of this relay could add the power measurements

Controls



esphome shelly-1-gen4-zigbee



[Add to dashboard](#)

Diagnostic



Identify

Press

[+2 disabled entities](#)

[Add to dashboard](#)

Activity



Today · October 8, 2026

- esphome shelly-1-gen4-zigbee Zha Event was fired

11:27:57 AM
- esphome shelly-1-gen4-zigbee Unavailable

11:27:57 AM
- Identify Pressed

11:27:57 AM
- esphome shelly-1-gen4-zigbee Off

9:26:15 AM
- esphome shelly-1-gen4-zigbee On

9:26:06 AM
- esphome shelly-1-gen4-zigbee Off

8:51:42 AM

Future versions

I have a PM version of the Shelly 1 Gen 4 (The red one). I believe the code can be adapted to expose these measurements. I will likely wait for the ESphome native zigbee cluster On/off is supported. Stay tuned to this repo for future developments